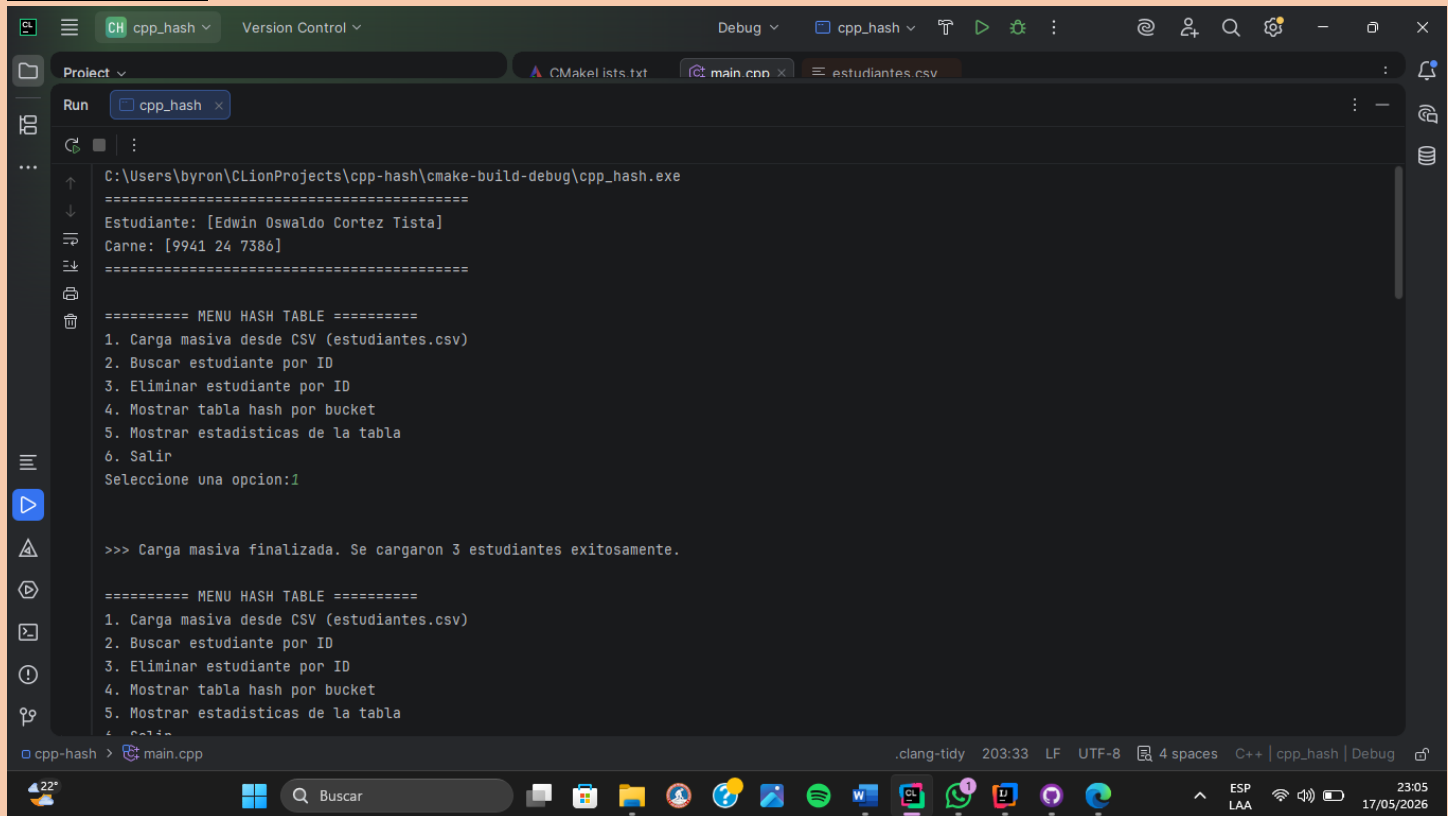


CODIGO EN C++:

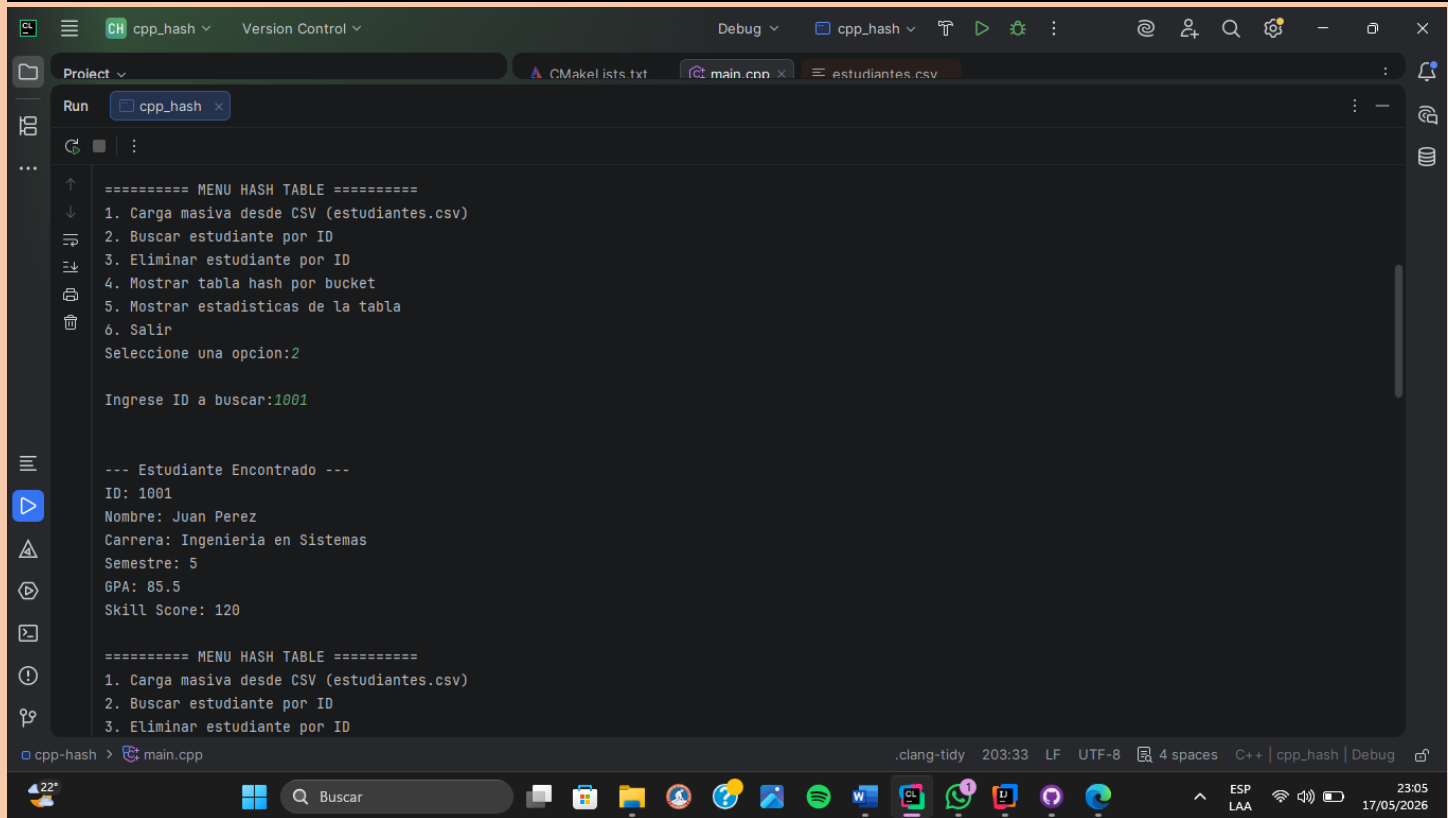


```
C:\Users\byron\CLionProjects\cpp-hash\cmake-build-debug\cpp_hash.exe
=====
Estudiante: [Edwin Oswaldo Cortez Tista]
Carne: [9941 24 7386]
=====

===== MENU HASH TABLE =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar tabla hash por bucket
5. Mostrar estadísticas de la tabla
6. Salir
Seleccione una opción:1

>>> Carga masiva finalizada. Se cargaron 3 estudiantes exitosamente.

===== MENU HASH TABLE =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar tabla hash por bucket
5. Mostrar estadísticas de la tabla
6. Salir
```



```
===== MENU HASH TABLE =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar tabla hash por bucket
5. Mostrar estadísticas de la tabla
6. Salir
Seleccione una opción:2

Ingrese ID a buscar:1001

--- Estudiante Encontrado ---
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
Skill Score: 120

===== MENU HASH TABLE =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
```

CH cpp_hashVersion Control

Debugcpp_hashT

ProjectCMakeLists.txtmain.cppestudiantes.csv

Runcpp_hash

1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar tabla hash por bucket
5. Mostrar estadísticas de la tabla
6. Salir
Seleccione una opcion:3

Ingrese ID a eliminar:1002

Estudiante eliminado correctamente.

===== MENU HASH TABLE =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar tabla hash por bucket
5. Mostrar estadísticas de la tabla
6. Salir
Seleccione una opcion:4

--- CONTENIDO DE LA HASH TABLE (POR BUCKET) ---
Bucket [0]: vacioNULL

clang-tidy203:33LFUTF-84 spacesC++ | cpp_hash | Debug

22°BuscarESP LAA23:0517/05/2026

CH cpp_hashVersion Control

Debugcpp_hashT

ProjectCMakeLists.txtmain.cppestudiantes.csv

Runcpp_hash

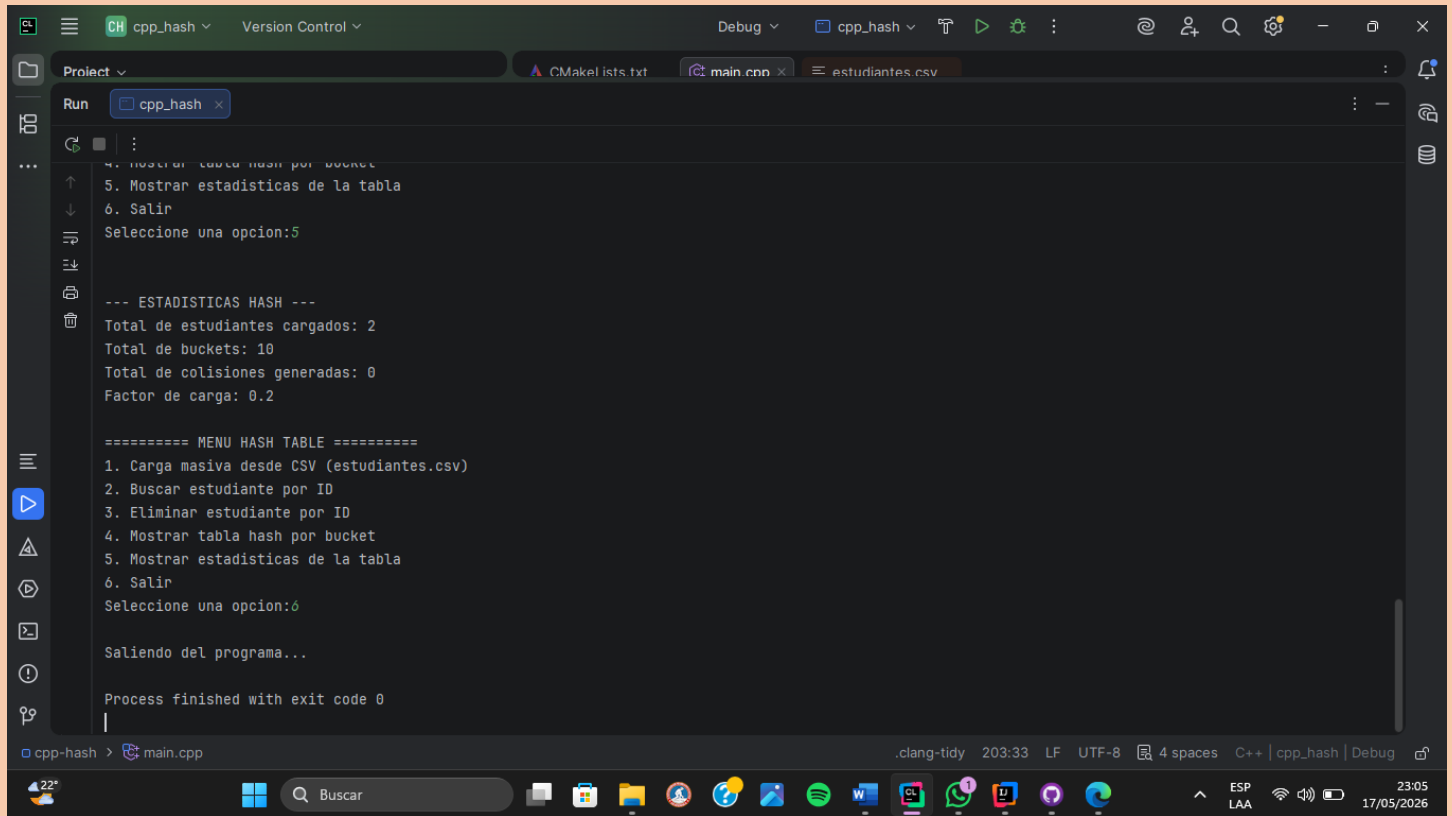
--- CONTENIDO DE LA HASH TABLE (POR BUCKET) ---
Bucket [0]: vacioNULL
Bucket [1]: (1001, Juan Perez, Ingenieria en Sistemas) -> NULL
Bucket [2]: vacioNULL
Bucket [3]: (1003, Carlos Ruiz, Ingenieria Civil) -> NULL
Bucket [4]: vacioNULL
Bucket [5]: vacioNULL
Bucket [6]: vacioNULL
Bucket [7]: vacioNULL
Bucket [8]: vacioNULL
Bucket [9]: vacioNULL

===== MENU HASH TABLE =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar tabla hash por bucket
5. Mostrar estadísticas de la tabla
6. Salir
Seleccione una opcion:5

--- ESTADISTICAS HASH ---
Total de estudiantes cargados: 2

clang-tidy203:33LFUTF-84 spacesC++ | cpp_hash | Debug

22°BuscarESP LAA23:0517/05/2026



Project

Run

Current File

Unlock Ultimate

Estudiante.java

Main.java

estudiantes.csv

```
===== MENU HASHMAP (JAVA) =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar estudiantes actuales (Evidencia HashMap)
5. Mostrar estadísticas
6. Salir
Seleccione una opcion: 3
Ingrese ID a eliminar: 1002

Estudiante eliminado.

===== MENU HASHMAP (JAVA) =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar estudiantes actuales (Evidencia HashMap)
5. Mostrar estadísticas
6. Salir
Seleccione una opcion: 4

=== ESTUDIANTES ACTUALES ===
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
Skill Score: 120
ID: 1003
Nombre: Carlos Ruiz
Carrera: Ingenieria Civil
Semestre: 3
GPA: 78.3
Skill Score: 95

===== MENU HASHMAP (JAVA) =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar estudiantes actuales (Evidencia HashMap)
5. Mostrar estadísticas
6. Salir
Seleccione una opcion: 5
```

actividad1 > src > Main > main 13:40 CRLF UTF-8 4 spaces 23:07 17/05/2026

Project

Run

Current File

Unlock Ultimate

Estudiante.java

Main.java

estudiantes.csv

```
=== ESTUDIANTES ACTUALES ===
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
Skill Score: 120
ID: 1003
Nombre: Carlos Ruiz
Carrera: Ingenieria Civil
Semestre: 3
GPA: 78.3
Skill Score: 95

===== MENU HASHMAP (JAVA) =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar estudiantes actuales (Evidencia HashMap)
5. Mostrar estadísticas
6. Salir
Seleccione una opcion: 5
```

actividad1 > src > Main > main 13:40 CRLF UTF-8 4 spaces 23:07 17/05/2026

```
===== MENU HASHMAP (JAVA) =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar estudiantes actuales (Evidencia HashMap)
5. Mostrar estadísticas
6. Salir
Seleccione una opcion: 5

=== ESTADISTICAS ===
Total estudiantes: 2

===== MENU HASHMAP (JAVA) =====
1. Carga masiva desde CSV (estudiantes.csv)
2. Buscar estudiante por ID
3. Eliminar estudiante por ID
4. Mostrar estudiantes actuales (Evidencia HashMap)
5. Mostrar estadísticas
6. Salir
Seleccione una opcion: 6
Saliendo del programa...

Process finished with exit code 0
```

Mi comparación: C++ (Manual) vs. Java (HashMap):

Para ser sincero trabajar este proyecto en los dos lenguajes me dejó ver las dos caras de la moneda en la programación, Por un lado, con C++ lo tuve que hacer todo a pie oea tuve que inventarme la función matemática para calcular el índice que es el módulo, crear los punteros de los nodos y programar la lógica de qué pasa cuando dos IDS que chocan en el mismo bucket y el Separate Chaining. Lo bueno de esto es que entendí perfectamente cómo funciona una tabla hash por dentro, y controlar cada byte de la memoria lo malo es que si te equivocas con un puntero, el programa se rompe por completo y encontrar el error es un dolor de cabeza y etresante.

Por otro lado, Java con su HashMap es estar en la gloria en cuanto a rapidez. no tuve que programar ninguna lista enlazada ni preocuparme por los choques de datos el lenguaje ya tiene todo eso resuelto y optimizado de fábrica. solo usé comandos sencillos como .put() para meter datos .get() para buscar y .containsKey() para ver si habían duplicados además, no me tuve que preocupar por liberar la memoria con destructores porque Java lo hace solo automaticamente.

En conclusión: Si necesito hacer algo rápido, seguro y sin complicarme la vida, me quedo con Java. Pero si lo que quiero es optimizar al máximo el rendimiento y entender al 100% cómo se manejan los datos en los chips de la computadora, C++ es el esencial, aunque cueste el doble de programar.